

Théo Castanié, Uriel Fellmann, Dylan Baric

Rapport de soutenance final Projet Rust



Introduction :

Après un long développement, notre projet de spectrogramme est arrivé à terme, sujet initialement choisi pour montrer l'ensemble des capacités de langage de programmation rust poussé en complexité.

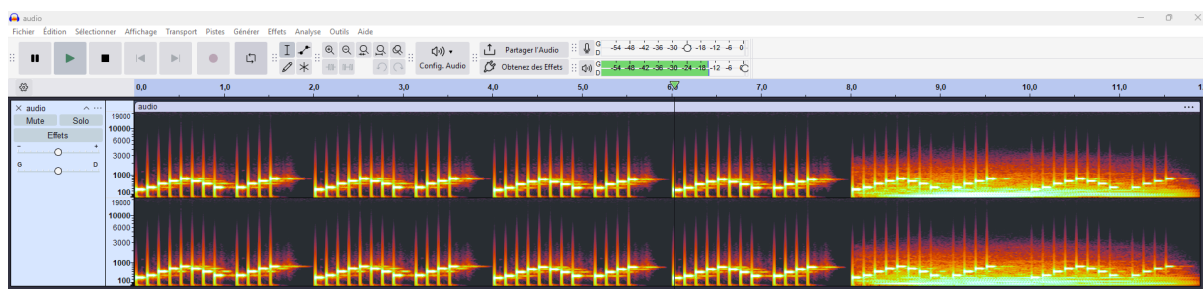
À la suite du rendu du 1er projet, toute l'équipe a dû se concentrer sur ce qu'il restait à faire , nous forçant à changer de focus de développement, tout le travail a plutôt été centré sur ce qui touchait au rendu, à l'optimisation et à l'expérience donnée.

Au fil du développement de ce 2e build, toute l'équipe a fait preuve de ses compétences pour rendre le meilleur produit, le plus optimisé possible.

Du côté de la sauvegarde et de la création du spectrogramme, nous avons repris notre précédent travail dans le but de parfaire l'image du spectrogramme pour la rendre plus précise mais également plus rapide à créer.

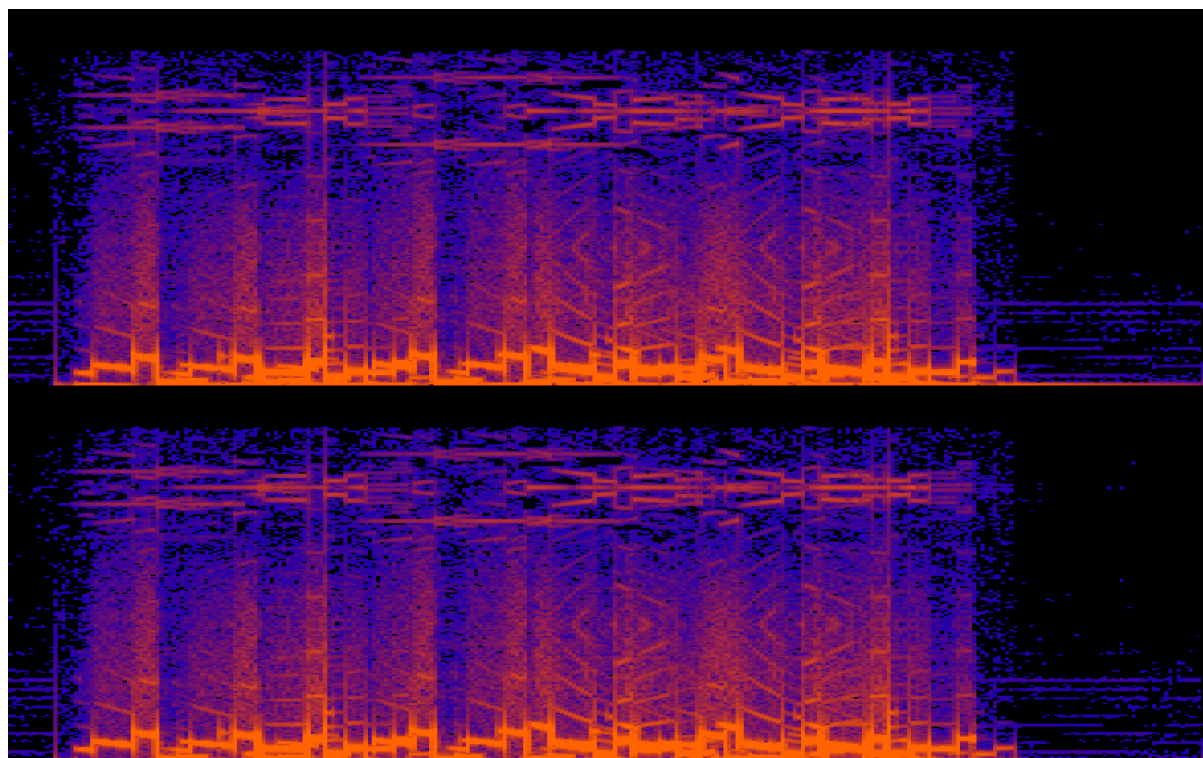
En effet jusque là notre programme renvoyait après presque une minute d'attente le spectrogramme demandé ce qui était très problématique. Étant donné que les fichiers avec lesquels on a testé notre créateur de spectrogramme étaient des fichiers .wav de quelques secondes cela voulait dire qu'avec des fichiers plus volumineux, le programme pouvait mettre encore plus de temps à s'exécuter. Il a donc fallu optimiser la création de spectrogramme et ce en changeant complètement notre approche pour analyser et créer le spectrogramme.

La première idée que nous avons eu fut de comparer le résultat renvoyé avec cette fois-ci non pas un site web mais avec l'application audacity. Cette application est très connue pour tout ce qui concerne les fichiers donc nous avons lancé rapidement l'application en question pour comparer les spectrogrammes que renvoyait audacity avec ceux de notre programme.



Audacity traitant les fichiers en .wav en stéréo.

Hormis la vitesse d'exécution, qui est à ce moment là encore un problème à régler, nous avons remarqué qu'en donnant les mêmes fichiers .wav l'application renvoyait constamment un 2 spectrogramme, tout deux empilé l'un sur l'autre. Cela a complètement changé notre approche pour la création du spectrogramme. Depuis le départ nous étions parti sur le traitement d'un fichier audio en mono là où Audacity les traitait en stereo. Nous avons donc modifié notre programme au départ pour faire en sorte à ce que l'on traite désormais tous nos fichiers .wav en stéréo.



Notre programme traitant un fichier .wav en stéréo plutôt qu'en mono.

Mais un second problème a fini par découler de ce raisonnement. En testant à nouveau notre programme et en observant plus attentivement les images renvoyées, nous nous sommes rendus compte que la plupart des fichiers n'avait pas besoin d'être traité en stéréo.

En effet, notre programme renvoyait constamment le même spectrogramme en haut comme en bas. Cela revenait donc à devoir créer deux fois le même spectrogramme inutilement en plus de perdre à nouveau du temps dans le renvoi du spectrogramme, ce qui n'est pas notre but.



Les images converties en .wav n'ont par exemple pas besoin d'être traitées en stereo.

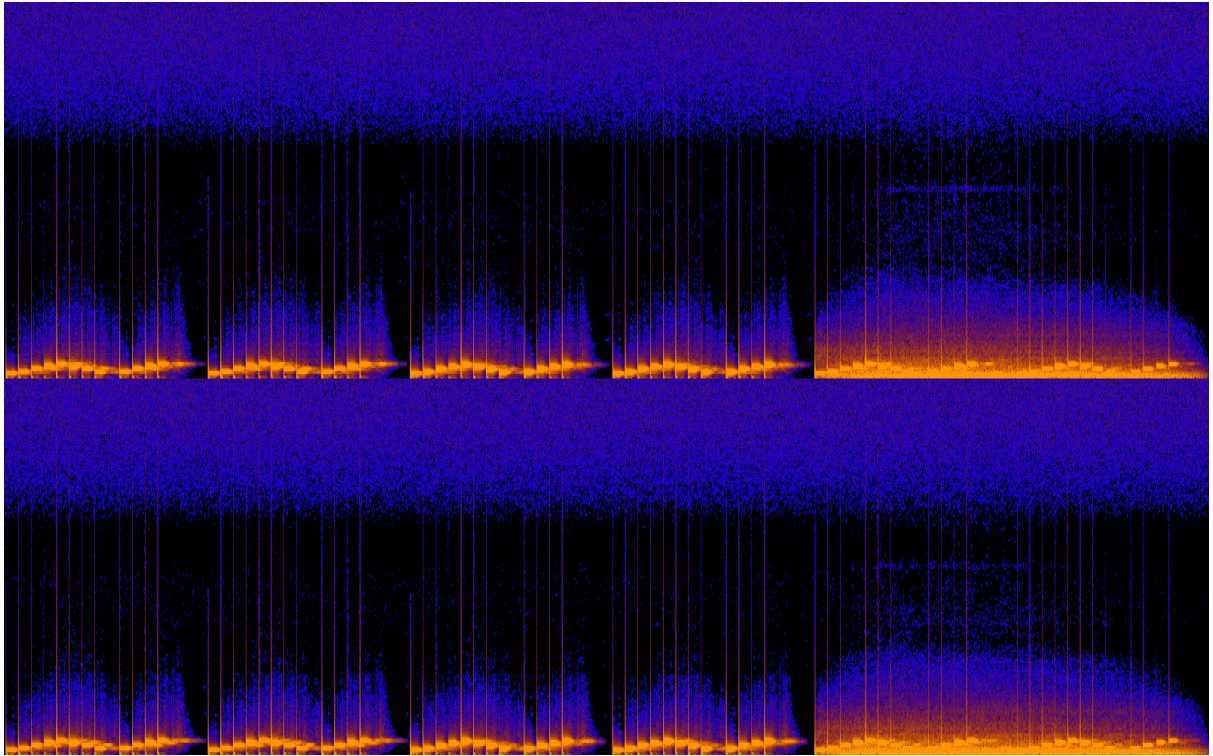
Nous avons alors modifié à nouveau notre analyseur de fichier audio pour qu'il détecte si les données à gauche et les données à droite qu'il est sur le point de transférer sont identiques. Une fois cette vérification faite, il indiquera alors au programme gérant la création du spectrogramme si le fichier traité doit être géré en mono ou stéréo. De là notre programme n'aura plus besoin de retirer des données identiques et pourra également renvoyer un spectrogramme bien plus lisible si le fichier audio donné a été traité en stéréo.



Les images converties en .wav réglées pour être de nouveau traitées en mono.

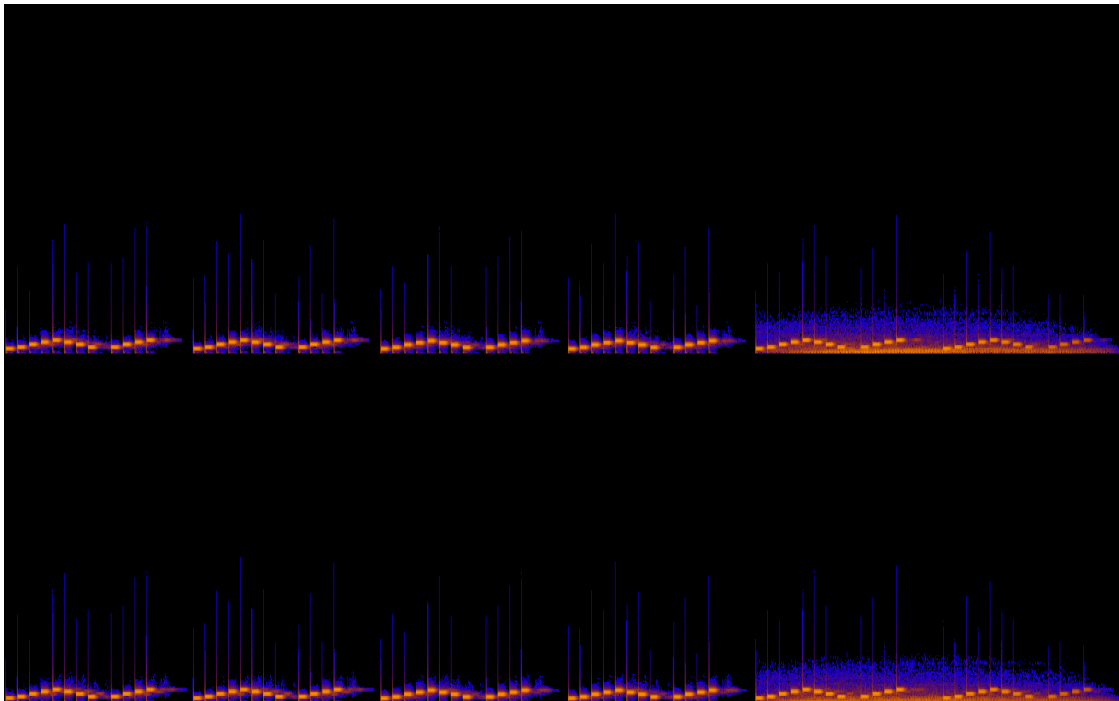
Une seconde optimisation que nous avons faite est au niveau du rendu du spectrogramme. Chaque image renvoyée est constamment renvoyée sous une dimension précise. Ainsi pour les fichiers plus volumineux, il sera bien plus difficile de voir avec précision le spectrogramme demandé.

Le problème dans le cas de la dimension de l'image a de nouveau été remarqué en comparant de plus près les résultats donnés par notre code avec ceux d'Audacity. Et justement notre spectrogramme était bien moins lisible. Cela était dû à la dimension choisie qui était tout simplement beaucoup trop grande pour n'importe quel fichier audio. En conséquence, notre spectrogramme était plus compact et donc moins lisible. De plus, une plus grande dimension et un débit minimal plus grand laisserait également dans l'image finale créer les bruits parasites. Des bruits inaudibles à l'oreille humaine, inutile lors de la lecture d'un spectrogramme.



Spectrogramme peu lisible en raison du bruit détecté.

Nous avons donc choisi une dimension d'image en apparence moins grande mais qui permet une meilleure lecture du spectrogramme du fichier audio demandé, en plus de traiter lors de l'attribution des couleurs pixel par pixel de notre case le cas des bruits parasites. Le spectrogramme était donc plus facile à lire et bien moins chaotique au premier coup d'œil.



Le spectrogramme du même fichier précédent, sans les bruits parasites.

Nous avons ensuite entamé notre quête vers la création du spectrogramme en temps réel sur notre site web.

Pour cela, il a fallu s'intéresser aux fonctionnalités web que propose Rust. Comme on l'a vu précédemment, il est possible de créer un site web avec Rust mais c'est très rudimentaire, cela reviendrait à créer un jeu vidéo en 2026 avec le langage assembleur.

Il y a donc différents modules permettant une transition entre les différentes parties. Le problème est que pour avoir un code directement implanté dans le site, il faut passer par WebAssembly. Et on n'avait clairement pas le temps d'apprendre de zéro un langage comme celui-ci. On a donc cherché d'autres alternatives.

On a trouvé un moyen un peu plus complexe mais fonctionnel : Créer un serveur local chez l'utilisateur qui permettrait d'exécuter le code avec ce que le site web envoie.

Le fonctionnement est simple : Avec le projet téléchargeable, il suffit de le lancer pour qu'un serveur local se crée, le site demande un fichier .wav qui est transmis au serveur local qui se charge donc d'en sortir un spectrogramme qu'il renvoie au site web et qui l'affiche. Pour cela, il fallait en premier lieu créer un fichier de transition entre le code, les rendus demandés et le site web : Il fallait un fichier javascript. Ce n'était pas le plus dur : il y a beaucoup de documentations à ce propos il fallait juste l'adapter un peu à son utilité précise dans notre cas.

Ensuite, il fallait créer un code rust qui crée un serveur local : D'autant plus, c'est une fonctionnalité que l'on n'a pas vu en cours. La documentation a été utile. On a pu trouver axum/tokio, qui est un bon système pour créer un routeur et "run" le fameux serveur. Plusieurs erreurs de compilation sont sans surprise apparues, c'est pour cela que le nombre de cas d'erreurs et d'informations qui sortent de la fonction principale est grand.

Pour fluidifier et mieux organiser nos codes, un generator a été créé dans le codex pour exécuter le code à la demande du serveur.

Il ne suffit plus qu'à générer le spectrogramme sur le site pour que la magie opère...

Résultat :

Spectrogramme généré !



Un problème important qui est apparu c'est que certains fichiers ne voulaient pas générer de spectrogramme. On a cherché jusqu'à ce que l'on se rende compte que c'était les fichiers les plus longs qui ne voulaient pas générer. En effet, il y avait une limite officieuse pour éviter les fichiers trop volumineux. Il a suffi de rajouter une ligne dans le code pour choisir le maximum.

```
.layer(DefaultBodyLimit::max(50*1024*1024)) //file limit to 50MB
```

On a préféré mettre une limite raisonnable de 50MB pour éviter de prendre des fichiers audios de plusieurs minutes, par gage de stabilité. Cela reste à confirmer malgré tout de jusqu'où la limite peut être.

Évidemment ce n'est pas un processus simple et notre objectif était de simplifier au maximum les étapes à faire pour exécuter le projet. Il a été préférable de mettre à disposition deux téléchargements différents pour avoir une version local de notre projet et une version site web. Entre les deux, il y a très peu de changements si ce n'est l'ajout du serveur local.

Grâce à cela, il suffit de faire Cargo run pour que tout soit opérationnel. Des consignes ont aussi été ajoutées sur le site pour favoriser la compréhension.

Consignes

1. Téléchargez le projet.
2. Dans la console, au niveau du dossier `img`, faites `./projet_rust` et attendez que le serveur local se lance.
3. Mettez votre fichier `.wav` sur le site. La limite du fichier est 50MB.
4. Appuyer sur "Générer le spectrogramme" et admirer le spectacle.

Ce fut d'ailleurs une épreuve supplémentaire pour rendre le site mieux agencé et plus agréable à ce propos.

Bilan et avis personnelles sur le projet:

Théo : J'ai beaucoup aimé travailler sur ce projet car il nous a en premier lieu permis de choisir d'une façon très libre ce que nous voulions faire. Par conséquent, cela a fait évoluer nos compétences en termes de gestion de ce type de projet. On a pu apprendre de nos erreurs et on a pu en apprendre plus sur ce langage de programmation atypique et aussi de pouvoir consolider nos acquis et la branche vers laquelle on veut se tourner pour notre avenir. À refaire !

Uriel : Travailler sur ce projet a été intéressant en raison de la liberté qu'il offrait sur le sujet. En effet, une majorité des décisions de développement nous étaient

laissées et devoir décider par nous même ce qu'il y avait à importer nous a forcé à faire des choix dans la documentation et à faire des choix sur l'optimisation assez difficiles. Je dirais que dans l'ensemble je me sens beaucoup plus confiant sur ma capacité à faire des choix dans un contexte de projet à long terme.

Dylan : J'ai personnellement apprécié dans l'ensemble travailler sur ce projet. Le traitement de fichier audio est un domaine que je connaissais peu auparavant alors travailler sur cet axe m'a permis d'en apprendre et de comprendre un peu plus le monde du traitement audio. De plus le fait que ce projet soit centré sur le Rust plutôt que le C/C# comme nous en avons eu l'habitude m'a permis de tester et d'enrichir mes connaissances en Rust d'une toute autre manière. Ainsi malgré les difficultés à parfois comprendre certaines choses, travailler sur un projet reposant sur moins d'acquis que précédemment fut en somme un défi particulièrement intéressant.

Conclusion :

Pour conclure, tout ce travail sur un projet à long terme, après tout ceux ayant été fait précédemment, nous a permis de voir notre progression sur le long terme (comparé au projet du S3 par exemple) et de voir notre progression sur le court terme en Rust.

Toute l'équipe est fière de son travail sur notre programme et est heureuse d'avoir pu avoir un exemple de ses compétences.